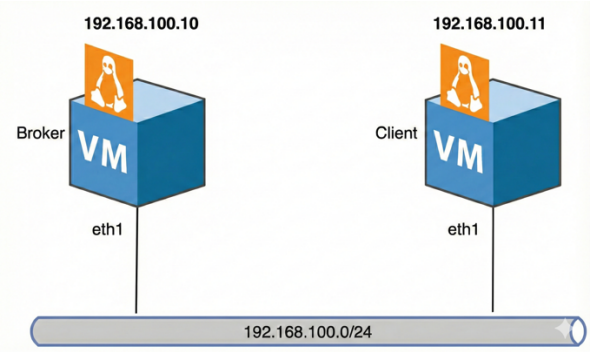
TP
SÉCURISATION D'UNE CONNEXION MQTT

Objectif terminal :
Sécuriser une connexion mqtt pour le client et le serveur.

Objectifs intermédiaires :

- Créer des machines virtuelles en utilisant KVM et vagrant.
- Créer un certificat SSL/TLS.
- Configurer un broker MQTT :
  - Configurer le MQTTS.
  - Implémenter les certificats.
  - Implémenter l'authentification par mot de passe.

TOPOLOGIE DU TP :



Nous allons utiliser Vagrant et KVM pour déployer deux machines virtuelles

CRÉATION DES MACHINES VIRTUELLES :

1. Lancer un terminal.

2. Télécharger le fichier Vagrantfile :

```
git clone https://github.com/bouhenic/FormationIOTSecu
cd FormationIOTSecu/mqtts
```

*Nous allons utiliser **KVM** et **Vagrant** pour déployer deux machines virtuelles (VMs). **KVM** est un hyperviseur qui permet d'exécuter des VMs sur un hôte LINUX, tandis que **Vagrant** est un outil d'automatisation qui facilite la gestion, la configuration et le provisionnement des VMs via des fichiers de configuration. Cette approche permet de déployer rapidement des environnements reproductibles et cohérents.*

```
cat  Vagrantfile
```

```
Vagrant.configure("2") do |config|
  # Image de base pour les deux machines
  config.vm.box = "debian/bookworm64"

  # --- CONFIGURATION COMMUNE (Provisioning) ---
  config.vm.provision "shell", inline: <<-SHELL
    export DEBIAN_FRONTEND=noninteractive
    sudo apt-get update
    # Installation des outils demandés
    sudo apt-get install -y mosquitto mosquitto-clients openssl
    # Clavier en français (console)
    localectl set-keymap fr
    echo 'KEYMAP=fr' > /etc/vconsole.conf
  SHELL

  config.vm.synced_folder ".", "/vagrant", disabled: true

  # --- MACHINE 1 : LE BROKER ---
  config.vm.define "mosquitto-broker" do |broker|
    broker.vm.hostname = "mosquitto-broker"
    broker.vm.network "private_network", ip: "192.168.100.10"

    broker.vm.provider :libvirt do |v|
      v.memory = 2048
      v.cpus = 2
      v.storage_pool_name = "images"
    end

    # Configuration spécifique pour autoriser les connexions distantes sur Mosquitto
    broker.vm.provision "shell", inline: <<-SHELL
      echo "listener 1883 0.0.0.0" > /etc/mosquitto/conf.d/external.conf
      echo "allow_anonymous true" >> /etc/mosquitto/conf.d/external.conf
      systemctl restart mosquitto
    SHELL
  end

  # --- MACHINE 2 : LE CLIENT ---
  config.vm.define "mosquitto-client" do |client|
    client.vm.hostname = "mosquitto-client"
    client.vm.network "private_network", ip: "192.168.100.11"

    client.vm.provider :libvirt do |v|
      v.memory = 2048
      v.cpus = 2
      v.storage_pool_name = "images"
    end
  end
end
```

3. Créer les machines virtuelles

```
vagrant up --provider=libvirt
```

4. Vérifier la création des machines virtuelles

```
vagrant status
```

```
Current machine states:

mosquitto-broker          running (libvirt)
mosquitto-client          running (libvirt)

This environment represents multiple VMs. The VMs are all listed
above with their current state. For more information about a specific
VM, run `vagrant status NAME`.
```

5. Se connecter à la VM broker

```
vagrant ssh mosquitto-broker
```

6. Lancer un second terminal.

7. Se connecter à la VM client

```
vagrant ssh mosquitto-client
```

CRÉATION DES CERTIFICATS SERVEUR :

Nous allons utiliser openssl. C'est une boîte à outils de cryptographie robuste pour le SSL/TLS, utilisée pour sécuriser les communications sur les réseaux informatiques. Elle offre des fonctionnalités pour la création de clés cryptographiques, de certificats, la gestion de l'authentification SSL/TLS, et la réalisation de diverses opérations de chiffrement.

1. Créer un certificat CA :

Une Autorité de Certification (CA) est une entité de confiance qui émet des certificats numériques pour valider l'identité des parties dans une communication sécurisée par SSL/TLS.

Le certificat va nous permettre d'auto-signer notre certificat serveur sans passer par un véritable CA. On utilise cette méthode en local ou en mode développement. En production, il faudra passer par un CA de confiance.

Depuis la VM broker, saisissez la commande openssl suivante :

```
openssl req -new -x509 -days 1826 -extensions v3_ca -keyout ca.key -
out ca.crt
```

```
Generating a RSA private key
....+++++
.....................................................................+++++
writing new private key to 'ca.key'
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:PARIS
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:
```

On choisit un password que l'on mémorise

FR

Rien ici

PARIS

Rien ici

Rien ici

Rien ici

Rien ici

- openssl req : Utilise le module req d'OpenSSL pour gérer les demandes de signature de certificat (CSR) et générer des certificats auto-signés.

- -new : Crée une nouvelle demande de signature de certificat (CSR).
- -x509 : Indique que l'on souhaite générer un certificat auto-signé plutôt qu'une simple CSR. Les certificats X.509 sont le standard pour les certificats SSL/TLS.
- -days 1826 : Définit la durée de validité du certificat à 1826 jours, soit environ 5 ans.
- -extensions v3_ca : Spécifie l'utilisation des extensions X.509 version 3 pour une autorité de certification, permettant d'ajouter des informations supplémentaires telles que les usages de clé, les politiques de certificat, etc.
- -keyout ca.key : Spécifie le fichier de sortie pour la clé privée générée (ca.key).
- -out ca.crt : Spécifie le fichier de sortie pour le certificat auto-signé généré (ca.crt).

2. Créer la clé privée pour le serveur :

```
openssl genrsa -out server.key 2048
```

```
Generating RSA private key, 2048 bit long modulus (2 primes)
.................+++++
.......................................................................+++++
e is 65537 (0x010001)
```

- genrsa : Indique à OpenSSL de générer une nouvelle paire de clés RSA. RSA (Rivest-Shamir-Adleman) est l'un des premiers algorithmes de cryptographie à clé publique et est largement utilisé pour sécuriser les transmissions de données.
- -out server.key : Spécifie le fichier de sortie (server.key) où la clé privée RSA générée sera sauvegardée. Ce fichier contiendra la clé privée en format PEM (Privacy Enhanced Mail), un format de fichier utilisé pour stocker et échanger des données cryptographiques.
- 2048 : Définit la taille de la clé en bits. Dans cet exemple, une clé de 2048 bits est générée. La taille de la clé est un facteur important pour la sécurité du chiffrement ; 2048 bits sont considérés comme sécurisés pour une utilisation actuelle, offrant un bon équilibre entre sécurité et performance.

3. Créer une demande de signature de certificat (CSR) :

```
openssl req -out server.csr -key server.key -new
```

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:PARIS
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Newton
Organizational Unit Name (eg, section) []:CIEL
Common Name (e.g. server FQDN or YOUR name) []:192.168.100.10
Email Address []:ciel@newton.fr

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

FR
Rien ici
PARIS
Votre organisation
Votre section
IP du broker
Un mail bidon
Rien ici

- -out server.csr : Spécifie le fichier de sortie (server.csr) où la demande de signature de certificat générée sera sauvegardée. La CSR contient des informations

d'identification du serveur (telles que le nom commun, l'organisation, le pays, etc.) ainsi que la clé publique du serveur.

- -key server.key : Indique le fichier contenant la clé privée RSA (server.key) qui sera utilisée pour générer la CSR. Cette clé privée ne sera pas incluse dans la CSR elle-même, mais la clé publique correspondante et les informations d'identification seront.

4. Signer la Demande de Signature de Certificat (CSR) et générer un certificat SSL/TLS signé:

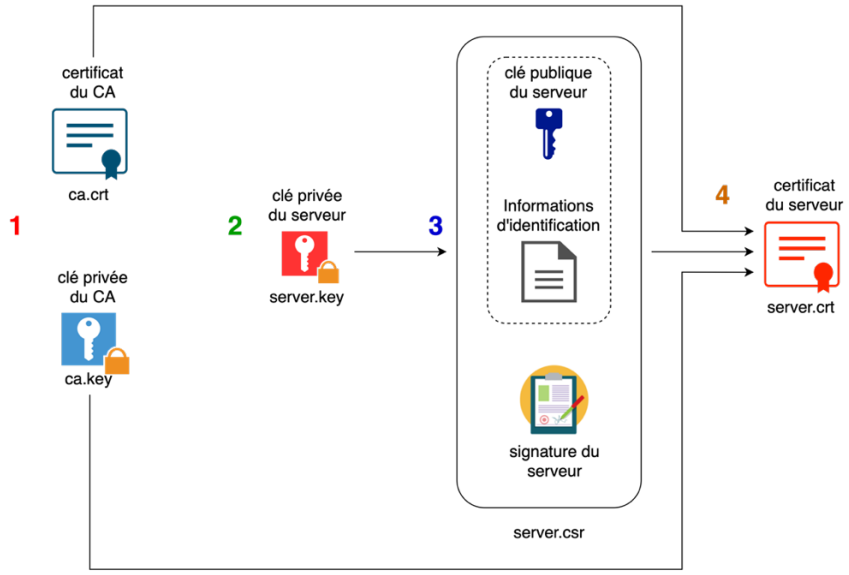
```
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -
CAcreateserial -out server.crt -days 360
```

```
Certificate request self-signature ok
subject=C = FR, ST = Some-State, L = PARIS, O = Newton, OU = CIEL, CN = 192.168.100.10, emailAddress = ciel@newton.fr
Enter pass phrase for ca.key:
```

Entrer ici le mot de passe défini lors de la création du certificat CA

- in server.csr : Spécifie le fichier d'entrée contenant la CSR. server.csr est le fichier qui contient la demande de certificat générée par ou pour le serveur, qui souhaite obtenir un certificat signé.
- CA ca.crt : Indique le certificat de l'Autorité de Certification (CA) utilisé pour signer la CSR. ca.crt contient le certificat public de la CA.
- CAkey ca.key : Spécifie la clé privée de l'Autorité de Certification (ca.key) qui correspond au certificat public spécifié par -CA. Cette clé privée est utilisée pour signer effectivement la CSR et générer le certificat signé.
- out server.crt : Définit le nom du fichier de sortie pour le certificat signé. Dans cet exemple, le certificat signé est sauvegardé dans server.crt.

EN RÉSUMÉ :



1 : `openssl req -new -x509 -days 1826 -extensions v3_ca -keyout ca.key -out ca.crt`

2 : `openssl genrsa -out server.key 2048`

3 : `openssl req -out server.csr -key server.key -new`

4 : `openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 360`

CONFIGURATION DU BROKER :

1. Modifier le fichier de configuration mosquitto.conf :

```
sudo nano /etc/mosquitto/mosquitto.conf
```

2. Ajouter les lignes suivantes en fin de fichier :

```
listener 8883
cafile /home/vagrant/ca.crt
certfile /home/vagrant/server.crt
keyfile /home/vagrant/server.key
```

3. Relancer le service mosquitto.

```
sudo systemctl restart mosquitto
```

4. Vérifier l'état running du service mosquitto

```
sudo systemctl status mosquitto
```

RECOPIE DU CERTIFICAT D'AUTORITÉ SUR LE CLIENT :

1. Editer le contenu du fichier ca.crt

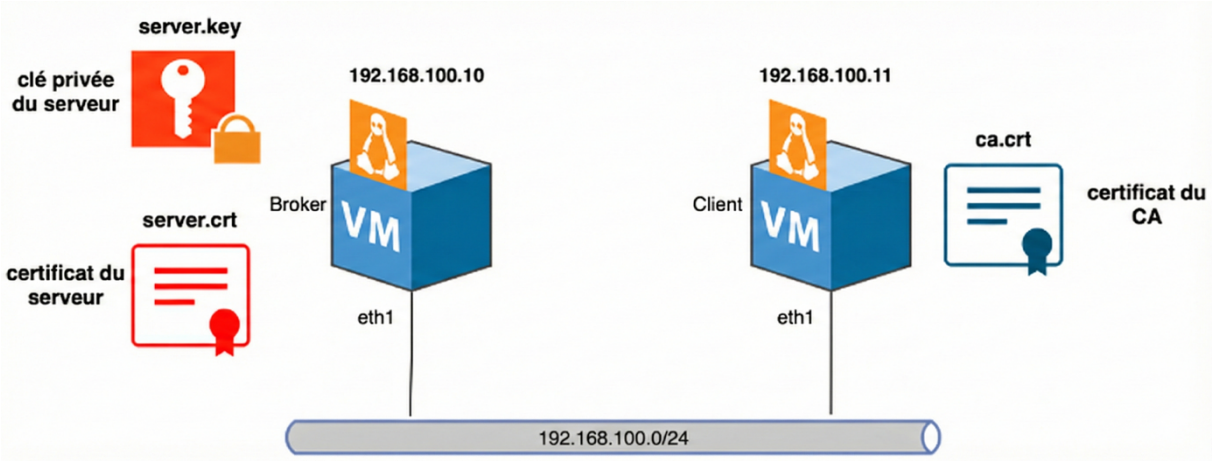
```
cat /home/vagrant/ca.crt
```

2. Copier son contenu dans le presse-papier et créer un fichier ca.crt sur le client

```
nano /home/vagrant/ca.crt
```

3. Coller le contenu du presse-papier dans fichier ca.crt édité précédemment sur le client.

TEST DU MQTTS:



1. S’abonner à un topic depuis la VM broker :

```
mosquitto_sub -h 192.168.100.10 -p 8883 --cafile /home/vagrant/ca.crt -t
your/topic
```

2. Publier un message depuis la VM client :

```
mosquitto_pub -h 192.168.100.10 -p 8883 --cafile /home/vagrant/ca.crt -t
your/topic -m "Hello world"
```

AUTHENTIFICATION DU CLIENT PAR MOT DE PASSE :

1. Modifier le fichier de configuration mosquitto.conf sur la VM broker :

```
sudo nano /etc/mosquitto/mosquitto.conf
```

2. Ajouter les lignes suivantes en fin de fichier :

```
allow_anonymous false
password_file /mosquitto_passwd
```

3. Créer des utilisateurs clients :

```
sudo mosquitto_passwd -c /mosquitto_passwd userclient
```

```
Password:
Reenter password:
```

```
sudo mosquitto_passwd  /mosquitto_passwd userbroker
```

4. Relancer le service mosquitto

```
sudo systemctl restart mosquitto
```

5. Vérifier les utilisateurs/mot de passe créé :

Tout se trouve dans le fichier mosquitto_passwd :

```
cat /mosquitto_passwd
```

```
userclient:$6$OvNJCOKGrIdI0xFf$He6Y7CTq6VBSmH9QW2OgogaUHKsuzH9LJPuvUCQKuivqgK3OOw75F4sbY/
1d5G/wmp7rAwhG61PJpm5098LxOA==
userbroker:$6$FIcGeh9rw19Zr6o+$ajleHiJlGAjlTlD279mQKsygnBUb55SpegREVE9m8KiWOMdg/DsnO+nYUS
Og9ayLcvFUszBeAgjnFjx6BAwtRw==
userclient2:$6$gtuGskWMC0SermQ9$SWK/SXSr4+Thoh1PKttV5tHK7+vJn96m6E0JEMC25EFHuq4wdtlf5zrdR
uKtMzlQttpxnXdiK4lFMIc57PLF/Q==
```

On voit les utilisateurs en clair et les mots de passe hachés.

userclient:6OvNJCOKGrIdI0xFf$He6Y7CTq6VBSmH9QW2OgogaUHKsuzH9LJPuvUCQKuivqgK3OOw75F4sbY/1d5G/wmp7rAwhG61PJpm5098LxOA==

userclient : C'est le nom d'utilisateur. Dans ce cas, l'utilisateur s'appelle userclient.

6OvNJCOKGrIdI0xFf$He6Y7CTq6VBSmH9QW2OgogaUHKsuzH9LJPuvUCQKuivqgK3OOw75F4sbY/1d5G/wmp7rAwhG61PJpm5098LxOA== : Ceci est le mot de passe hashé pour userclient. Le format du hash suit la convention utilisée par mosquitto_passwd, où :

- 6 indique le type de l'algorithme de hachage utilisé. 6 représente SHA-512, un choix courant pour le hachage des mots de passe en raison de sa sécurité.
- Le texte entre le premier et le second $ (OvNJCOKGrIdI0xFf dans cet exemple) est le "salt", une chaîne aléatoire ajoutée au mot de passe avant le hachage pour assurer que les hashages de deux mots de passe identiques soient différents.
- Le reste (He6Y7CTq6VBSmH9QW2OgogaUHKsuzH9LJPuvUCQKuivqgK3OOw75F4sbY/1d5G/wmp7rAwhG61PJpm5098LxOA==) est le mot de passe réellement hashé avec le salt.

TEST DE L'AUTHENTIFICATION PAR MOT DE PASSE

1. S'abonner à un topic depuis le broker :

```
mosquitto_sub -h 192.168.100.10 -p 8883 --cafile /home/vagrant/ca.crt -u
userbroker -P xxxxxxxxxx -t your/topic
```

2. Publier un message depuis le client :

```
mosquitto_pub -h 192.168.100.10 -p 8883 --cafile /home/vagrant/ca.crt -u
userclient -P xxxxxxxxxx -t your/topic -m "Hello world"
```